FACULTEIT INGENIEURSWETENSCHAPPEN
EN ARCHITECTUUR

Besturingssystemen: labo

W. Van den Breen

Bachelor in de industriële wetenschappen: informatica
Bachelor in de industriële wetenschappen: elektronica-ICT

Academiejaar 2025–2026

UNIVERSITEIT
GENT

Deel I: Inleiding

Lokaal inloggen op een Linux toestel kan op twee manieren gebeuren. Je kan inloggen op de grafische console of je kan inloggen op een van de vijf a zes tekstuele consoles. Veranderen van console kan via de toetsencombinaties CTRL-ALT-F1, CTRL-ALT-F2, … , CTRL-ALT-F7. Wanneer je Linux draait m.b.v. VirtualBox, dan moet je mogelijks gebruikmaken van de toetsencombinaties HOSTKEY-F1, HOSTKEY-F2, …, HOSTKEY-F7 waarbij de HOSTKEY standaard is ingesteld op de rechter CTRL-toets. Na het opstarten van Linux wordt aan iedere console een tty (terminal type) gekoppeld of in het geval van een grafische console een login manager zoals bv. gdm (GNOME Display Manager).

Bij Fedora Linux en aanverwante distributies zal de eerste console ingenomen worden door de grafische console. De overige tekstuele consoles vind je door te switchen met CTRL-ALT-Fx. Voor Debian, Ubuntu, … zal je merken dat de grafische console terug te vinden is op de zevende terminal.

Inloggen op meerdere terminals is mogelijk. Wanneer je correct inlogt op een tekstuele console kom je terecht in een BASH-shell. Bij het inloggen op de grafische console wordt naar alle waarschijnlijkheid een desktopomgeving zoals bv. GNOME, KDE, XFCE, … gestart. In die desktopomgeving kan je dan allerhande GUI-gebaseerde programma's starten naar analogie met MS Windows.

Voor deze labo's is het gebruik van een BASH-shell noodzakelijk. Op een tekstuele console beschik je onmiddellijk na het inloggen over een BASH-shell en op een grafische console zal je een terminal-emulator moeten starten.

1. De terminal

Log in als gebruiker *root* met als wachtwoord *e=mc**2* en start wanneer van toepassing in GNOME de terminal-emulator. Tik in het venster de opdracht "**stty -a"** in.

1. Welke toetsencombinatie heb je nodig om een runnend programma (een proces dus) te onderbreken?
2. Hoe kan je een runnend programma pauzeren? (Het hervatten van een programma zullen we later zien)
3. Wanneer een programma vraagt om gegevens in te typen, met welke toetsencombinatie kan je dan aangeven dat de invoer stopt?

Je kan vraag 1 en vraag 3 het best uitproberen m.b.v. het commando "**cat**". Deze opdracht kopieert alle invoer van het toetsenbord (standaard invoer) naar het scherm (standaard uitvoer).

Probeer dit nu ook uit met de opdracht "**mail root@localhost**". Hier zal eerst gevraagd worden om het onderwerp van het bericht in te geven, gevolgd door het bericht zelf. Wanneer je het einde van de invoer aangeeft, zal een mail verstuurd worden naar de gebruiker root op het lokale toestel. De gebruiker root kan zijn mail lezen door de opdracht **mail** uit te voeren zonder parameters. Hierdoor kom je terecht in een interactieve omgeving waar je een overzicht krijgt van alle berichten. Bij het intikken van een nummer dat naast een bericht staat, kan je de mail lezen. Het bericht verwijderen kan door het ingeven van de letter d gevolgd door het berichtnummer. De interactieve omgeving verlaten kan door het ingeven van de letter **q**.

4. Wanneer je op de commandolijn een aantal woorden intikt, hoe kan je dan het laatste woord verwijderen?

5. In een shell kan je aan "auto completion" doen door gebruik te maken van de tab-toets. Tik de letters "les" in, en tik vervolgens op de tab-toets. Je merkt dat er automatisch less verschijnt. Nu kan het zijn dat er nog opdrachten zijn die met less beginnen. Tik nogmaals op de tab-toets om te kijken welke opdrachten er met less beginnen. Een ander handigheidje is "reverse search" om eerder ingetypte commando's te suggereren tijdens het typen. Gebruik hiervoor "CTRL+r".

6. Linux telt enkele duizenden opdrachten met elk nog een variabel aantal opties. De opdracht zelf wordt doorgaans beperkt tot enkele letters die de afkorting vormen van een woord dat de functie van de opdracht deels omschrijft. Zo staat het commando "**ls"** voor "list**"**, "**mkdir**" voor "make directory", het commando "**wc"** voor "word count**"**, het commando "**cc"** voor "C-compiler**"**, het commando "**dd**" voor "convert and copy**"**. Bij het laatste voorbeeld is er een conflict omdat de afkorting cc reeds gebruikt wordt voor de C-compiler. Als oplossing heeft men de volgende letter van het alfabet gebruikt.

   "Auto completion" voor commando's werd reeds in het vorige punt naar voor gebracht maar geldt ook voor de bijhorende opties. Geef op de commandolijn het commando "**ls -**" in en gebruik vervolgens de tab-toets. Je merkt dat er een pak opties verschijnen die voorafgegaan worden door twee mintekens. Onder Linux kan je veelal opties op de commandolijn meegeven in twee formaten. Het korte formaat bestaat uit één letter en wordt voorafgegaan door één minteken. Dezelfde optie kan je ook meegeven d.m.v. het lange formaat. In dat geval zal de

optie voorafgegaan worden door twee opeenvolgende mintekens. Wanneer je de keuze hebt tussen de twee formaten zal je merken dat de korte variant de voorkeur geniet.

Zo kan het commando "**wc --words --lines /etc/passwd**" ook geschreven worden als "**wc -w -l /etc/passwd**" en zelfs als "**wc -wl /etc/passwd**". De opdracht toont het aantal lijnen en het aantal woorden uit het bestand /etc/passwd.

2. Gebruik van enkele eenvoudige opdrachten

    1. Met de opdracht man kan je alle info over een welbepaalde opdracht achterhalen. Zo zal de opdracht "**man ls**" de info tonen van de ls-opdracht.

       Man toont de informatie a.d.h.v. een pager, een programma dat de informatie niet alleen op het scherm toont maar dat ook gebruikersinteractie toelaat. Zo kan je scrollen, zoeken naar bepaalde woorden, etc. Standaard is de pager ingesteld op het commando **less**. Dit betekent dat wanneer je wil weten hoe je voorwaarts moet zoeken in een manpagina, je dit moet gaan zoeken bij het commando less en niet bij het commando man. Vraag de manpagina op van het commando less en ga na hoe je een tekst die met less wordt getoond kan afsluiten. Hoe kan je voorwaarts zoeken in een manpagina?
    2. Hoe kan je zorgen dat er bij het zoeken in een manpagina geen rekening wordt gehouden met het verschil tussen hoofd- en kleine letters?
    3. Bekijk de manpagina van het commando man en ga na hoeveel secties er gekend zijn.
    4. Wanneer je de opdracht "**man read**" intikt krijg je de info te zien van het commando read. Er is echter ook een systeemaanroep read aanwezig. Hoe kan je aan man meegeven dat je niet de info wenst te zien van het commando read maar wel van de systeemaanroep read?
    5. Met de opdracht "**ls"** krijg je van een directory een overzicht van alle bestanden en subdirectories te zien. Wanneer je geen directory opgeeft, wordt de huidige werkdirectory genomen. Wat doen de opties **-l** en **-h**?
    6. Bekijk met "**ls /"** de inhoud van de hoofddirectory.
    7. In punt 5 en 6 heb je gemerkt dat de uitvoer voorzien wordt van kleuren. Zo worden bv. directories in het donkerblauw gekleurd en symbolische links in het lichtblauw. Dit gedrag is te wijten aan het feit dat er voor ls een alias gedefinieerd is die telkens ls vervangt door "**ls --color=auto**". Ga met de opdracht "**alias**" na welke andere aliassen er bestaan.
    8. Met de opdracht "**cd**" kan je van werkdirectory veranderen. Wanneer je geen directorynaam opgeeft wordt je home-directory de nieuwe werkdirectory. Voer "**cd /tmp**" uit en keer daarna terug naar je home-directory.
    9. Een directory aanmaken kan via de opdracht "**mkdir**". Maak in je home-directory een directory aan met als naam 'c' waar je toekomstige C-programma's naartoe kan kopiëren.

10. Een bestand kopiëren gebeurt via de opdracht “**cp**”. De eerste parameter is de bron, de tweede de bestemming. Een bestand verplaatsen doe je via de opdracht “**mv**”.

Deel II: Compileren in de Shell

In Linux is het vrij eenvoudig om een C of C++ programma te compileren zonder gebruik te maken van een IDE. Het enige wat je nodig hebt is een eenvoudige tekstverwerker. In een grafische omgeving kun je bvb. **Visual Studio Code**, **Atom, Neovim** gebruiken. In tekstuele consoles is **nano** eenvoudig te gebruiken, terwijl **vi** en **vim** … al wat geavanceerder zijn.

Open in BASH een tekstverwerker (nano, vi, vim, neovim, …) naar keuze en tik daarin onderstaand codefragment:

```
#include <stdio.h>

int
main(){
        printf("Hello world!");
        return 0;
}
```

Sla dit bestand vervolgens op als hello.c (let op de extensie). Om nu dit bestand te compileren voer je op de commandolijn "**cc hello.c -o hello**" of "**make hello**" uit. Na het succesvol compileren kan je bestand uitvoeren door op de commandolijn "**./hello**" in te tikken. Bemerk dat je voor het uitvoerbaar bestand de "./" niet mag vergeten!

Wanneer je met de opdracht "**file**" informatie over het uitvoerbaar bestand opvraagt, zal je merken dat er gebruikgemaakt wordt van **shared libraries** en dat het uitvoerbaar bestand dus niet alle code bevat om de string "Hello world!" naar het scherm te schrijven. Is dit niet de bedoeling, dan kan je bij het compileren de compileroptie "**-static**" opgeven (installeer hiervoor de static versie van de C API via **yum install glibc-static**). Hierdoor wordt alles statisch gelinkt en krijg je dus een uitvoerbaar bestand dat alle code bevat om de gegeven tekst naar het scherm te schrijven.

Opdracht

Compileer bovenstaand bestand dynamisch en statisch en bekijk het verschil in grootte. De grootte van een bestand kan je het gemakkelijkst achterhalen met de opdracht "**du**" of met de opdracht "**ls**". Maak in beide gevallen handig gebruik van de optie "-h" om de bestandsgrootte in bytes, KB, MB, … te krijgen.

Deel III: Profiler programma's

Bij het schrijven van programma's zijn er een aantal zaken waar je toch moet op letten. Zo mag bv. de geschreven code geen memory leakage vertonen, mag de uitvoeringstijd niet te groot zijn en moet het aantal systeemaanroepen tot een minimum worden beperkt. Onder Linux zijn er een aantal profiler programma's waarmee je uitvoerbare bestanden kan onderzoeken.

Valgrind

Valgrind is een tool die in zijn meest eenvoudige vorm onderzoekt of de geschreven software memory leakage vertoond.

Opdracht

Schrijf een C-programma dat via malloc een tabel van 2000 gehele getallen aanmaakt en deze getallen daarna gewoon uitschrijft. Je hoeft bewust het in beslag genomen geheugen niet vrij te geven. Compileer het programma en voer het daarna uit d.m.v. "**valgrind ./prog**". Herschrijf nu het programma waar je het gealloceerde geheugen vrijgeeft en controleer opnieuw met valgrind of het programma nog geheugenlekken vertoond.

Time

Om na te gaan hoeveel CPU-tijd een bepaald programma in beslag heeft genomen kan je de opdracht "**time**" gebruiken. Het commando time geeft standaard drie timestamps terug. Het betreft, de totale tijd tussen het starten en het stoppen van het proces, de CPU-tijd voor het uitvoeren van code in USER-mode en tot slot de CPU-tijd voor het uitvoeren van code in de KERNEL-mode. De allereerste tijdsaanduiding is eigenlijk de minst interessante omdat een proces doorgaans niet in één keer wordt uitgevoerd maar door het besturingssysteem verschillende keren wordt onderbroken.

Opdracht

Herneem het programma dat je daarnet hebt geschreven en voer het nu uit d.m.v. "**time ./prog**".

Strace

Soms kan het interessant zijn om te weten welke systeemaanroepen een programma gebruikt. Bekijk de uitvoer van "**strace cat /etc/passwd**". Dit geeft een sequentieel overzicht van alles wat bij het uitvoeren van "cat /etc/passwd" gebeurt. Wanneer je een overzicht wil krijgen van welke systeemaanroepen er gebruikt worden en hoeveel keer ze werden opgeroepen, dan kan je aan strace de optie "**-c**" meegeven.

Overige interessante hulpprogramma's:

nm

Met het commando nm kan je symboolinformatie opvragen van uitvoerbare bestanden en libraries. Wil je weten van welke functies de Linux C API (glibc) de uitvoerbare code bevat? Voer in een terminal dan **nm /lib/libc.so.6** uit en alle lijnen waar een t of T bij vermeld staat zijn C-functies waarvan je de code via deze bibliotheek kan vinden.

ldd

Met ldd kan je van ieder uitvoerbaar bestand dat dynamisch gelinkt werkt opvragen van welke shared libraries het afhankelijk is.

Objdump

Wil je een uitvoerbaar bestand terug omzetten naar assembleertaal, dan kan je handig gebruikmaken van objdump met de optie -d (disassembly). Wil je de uitvoer zien in Intel syntax, voeg dan gerust de optie **-M intel** toe. Bij een uitvoerbaar bestand is het entrypoint, het punt waar het besturingssysteem code begint uit te voeren, de subroutine **_start**. Vanuit het oogpunt van een programmeur is dit main maar besef dat de eerste instructie die door het OS zal uitgevoerd worden niet de eerste lijn code uit de main-functie is.

readelf

Het standaard binaire formaat voor elk Unix-achtig besturingssysteem is ELF of Executable and linking format. De header bevat heel veel informatie die je kan uitlezen met readelf en dat gevolgd door de optie -a. Het entrypoint zal er in vermeld staan en zoals in de vorige paragraaf reeds geschreven, komt dit overeen met de subroutine _start.

Deel II: I/O-Systeemaanroepen

Alle systeemaanroepen die een verwijzing nodig hebben naar een open bestand, maken gebruik van een **file descriptor**. Een file descriptor is doorgaans een klein positief geheel getal dat kan verwijzen naar gewone bestanden maar ook naar directories, sockets, pipes, …. Alles wat eigenlijk op het bestandssysteem bestaat kan worden geopend en kan aan een file descriptor worden gekoppeld. Bij het starten van een programma zullen drie file descriptors van de shell worden overgeërfd. De drie file descriptors verwijzen respectievelijk naar het **standaard invoerkanaal**, het **standaard uitvoerkanaal** en het **standaard foutenkanaal**. Onderstaande tabel geeft een overzicht.

File descriptor	Doel	POSIX-naam	stdio-stream
0	Standaard invoerkanaal	STDIN_FILENO	stdin
1	Standaard uitvoerkanaal	STDOUT_FILENO	stdout
2	Standaard foutenkanaal	STDERR_FILENO	stderr

Een bestand openen gebeurt via de systeemaanroep **open(pathname, flags, mode)** die een geheel getal, de file descriptor dus, teruggeeft of -1 wanneer er een fout optreedt. De eerste paramater die je moet opgeven is het pad naar het bestand. De tweede parameter bevat een aantal statusvlaggen die bij het openen van het bestand worden geraadpleegd. Wanneer je een bestand wil openen om er uitsluitend van te lezen, geef je als flags *"O_RDONLY"* op. Wanneer je naar een bestand wil schrijven en er ook van wil lezen is de flags-parameter gelijk aan *"O_RDWR"*. Combineren van statusvlaggen gebeurt via de logische |-operator. Zo kan je een bestand openen om er naar te schrijven en tevens opgeven dat het moet worden aangemaakt wanneer het niet zou bestaan. De flags-parameters is in dit geval "*O_WRONLY | O_CREAT*". De combinatie *"O_WRONLY | O_CREAT | O_APPEND"* betekent dat het bestand zal worden aangemaakt wanneer het niet bestaat, dat er enkel naar geschreven zal worden en dat alle data die er naartoe wordt geschreven achteraan zal worden toegevoegd. De O_APPEND vlag verzekert dus dat indien het bestand reeds zou bestaan, de bestaande inhoud niet zal worden overschreven.

De laatste parameter van de systeemaanroep open is optioneel en is enkel van belang wanneer de O_CREAT vlag is opgegeven.

Om van een file descriptor te lezen gebruik je de systeemaanroep **read(fd,buffer,count)**. Buffer is een void-pointer naar een geheugenlocatie waar de gelezen bytes zullen worden weggeschreven. Count is het aantal bytes die je van de opgegeven file descriptor wil lezen. De read-systeemaanroep geeft het aantal gelezen bytes terug en -1 wanneer er bij het lezen een fout optreedt.

Met de **write(fd,buffer,count)** systeemaanroep kan je bytes van een geheugenlocatie naar een filedescriptor schrijven. De parameters zijn gelijkaardig aan die van de read-systeemaanroep en ook de return-waarde is hetzelfde, i.e. het aantal weggeschreven bytes of -1.

Tot slot kan je met de systeemaanroep **close(fd)** een filedescriptor sluiten.

Opdrachten

1. Schrijf een C-programma dat een bestand van ongeveer 10MB aanmaakt met willekeurige lettertekens gelegen in het gesloten interval [a..z].
2. Tot nog toe werd er niets gezegd over de optimale buffergrootte. De bedoeling is een C-programma te ontwikkelen dat het hierboven aangemaakte bestand verschillende keren inleest en dit met buffergroottes van 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096 en tot slot 8192 bytes. De uitvoer van het programma moet er ongeveer zo uitzien:

```
BUF_SIZ=   1    Time=3.60
BUF_SIZ=   2    Time=1.79
BUF_SIZ=   4    Time=0.98
BUF_SIZ=   8    Time=0.48
BUF_SIZ=  16    Time=0.24
BUF_SIZ=  32    Time=0.12
BUF_SIZ=  64    Time=0.06
BUF_SIZ= 128    Time=0.03
BUF_SIZ= 256    Time=0.02
BUF_SIZ= 512    Time=0.01
BUF_SIZ=1024    Time=0.00
BUF_SIZ=2048    Time=0.00
BUF_SIZ=4096    Time=0.00
BUF_SIZ=8192    Time=0.00
```

Om de tijd tussen het starten het stoppen met lezen op te meten kan je gebruikmaken van onderstaande code:

```
double start=clock();
…
double time=(clock()-start)/CLOCKS_PER_SEC;
```

De clock-functie en de macro CLOCKS_PER_SEC vind je in de bibliotheek pthread.h.

3. Doe nu hetzelfde maar voor de write-systeemaanroep. Maak voor iedere verschillende buffergrootte een bestand aan van ongeveer 10MB (cfr. vraag 1). Nadat je de tijd voor een gegeven buffergrootte hebt opgemeten moet je vanzelfsprekend het aangemaakte bestand terug verwijderen. Een bestand verwijderen kan je doen m.b.v. de **unlink**-systeemaanroep.
4. Herneem vraag 3 maar voeg aan de flags-parameter de *O_SYNC* vlag toe. Dit laatste zorgt ervoor dat er noch in USER-mode noch in KERNEL-mode zal worden gebufferd en dat de bytes rechtstreeks naar de schijf zullen worden geschreven.

   Ter info, de vaste schijf beschikt om performantieredenen over een bepaalde hoeveelheid write-back-cache-geheugen waar gegevens van I/O- schrijfopdrachten tijdelijk worden bewaard. Dit betekent dat er bij een eventuele spanningsonderbreking wel degelijk gegevens kunnen verloren gaan. Bij programma's, zoals gegevensbanken, waar gegevensverlies nefast is, wordt er aangeraden om "write-caching" uit te schakelen. Dit kan je doen door in een shell de opdracht "**hdparm -W0 /dev/sda**" uit te voeren.
5. Bestudeer gronding de werking van de shell-opdracht **cat** en schrijf in C een eigen versie van cat. Bekijk bv. wat er gebeurt wanneer je de opdracht "**cat /etc/passwd - /etc**" opgeeft of wanneer cat geen argumenten meekrijgt. Wanneer je een directory opgeeft als argument, geeft cat een foutmelding. Dit gedrag hoef je niet na te bootsen.
6. Schrijf een eigen versie van de opdracht **cp** waar twee argumenten op de opdrachtlijn worden verwacht. Het eerste argument is het bronbestand en het tweede het doelbestand. Wanneer de eerste parameter een directory is, wordt een foutboodschap naar het scherm geschreven en stopt het programma met exit-status 1. Wanneer het tweede argument een directory is, wordt opnieuw gestopt met een passende foutboodschap en met exit-status 1.

   Om na te gaan of een argument een directory is, kan je gebruikmaken van de systeemaanroep **stat**. Om een programma te stoppen met exit-status 1 maak je best gebruik van de bibliotheekfunctie **exit** die dan achter de schermen de systeemaanroep **_exit** aanroept.
7. Schrijf een C-programma met als naam *watchfile.c* dat één argument, een bestand, op de opdrachtlijn verwacht. Het programma loopt in een oneindige lus en schrijft telkens een boodschap naar het scherm wanneer het bestand dat op de opdrachtlijn werd meegegeven werd gewijzigd. Wanneer er geen argument werd opgegeven of het argument is geen gewoon bestand wordt een foutboodschap getoond en wordt het programma afgesloten met exit-status 1.

Deel III: Processen en POSIX-threads

Een nieuw proces aanmaken gebeurt met de systeemaanroep **fork()**. De systeemaanroep maakt een nieuw proces aan, het *kind*, dat bijna een exacte kopie is van het proces dat de systeemaanroep heeft gebruikt, het *ouderproces*. Na het beëindigen van de systeemaanroep beschik je over twee processen, het kind en het ouderproces, die *beide* de uitvoering verderzetten bij het terugkeren van de fork()-systeemaanroep. Beide processen beschikken over dezelfde programmatekst maar hebben elk hun eigen kopie van de stapel, de heap, en het datasegment. Initieel bevat de stapel, de heap, en het datasegment van het kindproces dezelfde informatie als de overeenkomstige segmenten van het ouderproces. Om in een programma het onderscheid te kunnen maken tussen het kind en het ouderproces moet je weten dat de systeemaanroep fork() twee verschillende return-waardes heeft. In het kindproces geeft fork() de waarde 0 terug, terwijl in het ouderproces het proces-ID van het nieuwe aangemaakte kindproces wordt teruggegeven.

Opdrachten

1. Gegeven onderstaande code:

```
int main(int argc, char **argv){
    …
    fork();
    fork();
    fork();
    …
}
```

   Voorspel hoeveel kindprocessen er zullen worden aangemaakt zonder de code uit te voeren.
2. Schrijf een programma dat drie kindprocessen aanmaakt en zorg ervoor dat ieder kindproces zijn proces-ID naar het scherm schrijft en daarna stopt. Het proces-ID kan je m.b.v. de systeemaanroep **getpid()** opvragen en een proces kan je beëindigen met de functie **exit(int exitstatus)**. De exitstatus van een correct beëindigd proces is steeds 0 terwijl een waarde verschillend van 0 duidt op een fout.

In bovenstaande opdracht kan je nog met een eenvoudige if/else-structuur het onderscheid maken tussen wat het ouderproces en wat het kindproces moet uitvoeren. Het kindproces moet immers alleen maar zijn proces-ID naar het scherm schrijven en zichzelf beëindigen. Wanneer het gaat om meer dan een paar lijnen code, is het interessanter om een nieuw programma te ontwikkelen, eventueel zelfs in een andere programmeertaal, dat je onmiddellijk na de fork()-systeemaanroep uitvoert. Hetzelfde geldt wanneer het kindproces een programma moet uitvoeren dat reeds bestaat en dat dus niet door jou nog moet worden ontwikkeld.

Een nieuw programma uitvoeren gebeurt met de systeemaanroep **execve(const char* filename, char *const argv[], char *const envp[])**. Execve zal het tekstsegment, het datasegment, de stapel en de heap van het huidige proces overschrijven met de gegevens van het uitvoerbaar bestand *filename*. Argumenten voor het nieuwe programma worden als tweede parameter *argv* opgegeven. Als laatste parameter *envp* kan je een tabel van strings, in de vorm van naam-waardeparen, doorgeven aan het nieuwe programma.

Bemerk dat bij de oplijsting van systeemaanroepen met **strace** execve op de eerste lijn staat. Concreet wil dit zeggen dat van het shellproces een kopie gemaakt wordt en dat onmiddellijk daarna het proces wordt overschreven met de inhoud van de opdracht die je op de opdrachtlijn hebt ingetikt.

Naast het rechtstreeks gebruik van de systeemaanroep execve, bestaan er een aantal *bibliotheek-functies* die allemaal na een aantal tussenstappen execve zullen aanroepen. Deze functies beginnen allemaal met het woord exec gevolgd door één of twee letters. Het verschil met execve zit hem in het aantal parameters en de manier waarop de argumenten aan het nieuwe programma worden doorgegeven. Hieronder vind je een overzicht.

Functie	Specificatie van de programmanaam	Specificatie van de argumenten	Environment-argumenten
Systeemaanroep			
execve	Volledige padnaam	Tabel van C-strings	envp-argument
Bibliotheekfuncties			
execle	Volledige padnaam	Opsomming van C-strings	envp-argument
execlp	Bestandsnaam(PATH wordt geraadpleegd)	Opsomming van C-strings	Niet van toepassing
execvp	Bestandsnaam(PATH wordt geraadpleegd)	Tabel van C-strings	Niet van toepassing
execv	Volledige padnaam	Tabel van C-strings	Niet van toepassing
execl	Volledige padnaam	Opsomming van C-strings	Niet van toepassing

3. Schrijf een C-programma **writestring.c** dat het proces-ID naar het scherm schrijft gevolgd door de string die als enige parameter wordt meegegeven en vervolgens 10 seconden wacht vooraleer te eindigen.
    1. Herschrijf nu opdracht 2 waarbij het kindproces de systeemaanroep **execv** gebruikt om **"writestring hello"** uit te voeren.
       Opgelet: In het ouderproces moet je wachten tot wanneer het kindproces klaar is, waarna je nog een boodschap naar het scherm schrijft. Dit doe je door gebruik te maken van de systeemaanroep **waitid** of **waitpid**.

2. Idem als deel 1 maar maak nu gebruik van de systeemaanroep **execl**. De laatste C-string-parameter moet **(char *)0** zijn om het einde van de opsomming aan te geven. Bemerk dat gewoon 0 schrijven niet voldoende is en zelfs fout is. Wanneer de grootte van een int verschillend is van de grootte van een char *, zal het aantal argumenten dat doorgegeven wordt aan execl verkeerd zijn.

Interprocescommunicatie, afgekort IPC, in Unix en dus ook in Linux gebeurt veelal door gebruik te maken van pipes. Een **unnamed pipe** of **anonymous pipe** wordt gebruikt wanneer twee processen rechtstreeks van elkaar afstammen. Processen die geen gemeenschappelijke band hebben, moeten gebruikmaken van **named pipes**. Een pipe is een **unidirectionele communicatielijn** tussen twee processen. Wanneer twee processen gegevens willen uitwisselen in beide richtingen heb je twee pipes nodig!

Om een anonieme pipe aan te maken kan je gebruikmaken van de systeemaanroep **pipe(int fd[2])** die -1 teruggeeft wanneer de pipe niet kan worden aangemaakt. In het andere geval zal fd twee filedescriptors bevatten, nl. fd[0] om van de pipe te lezen en fd[1] om naar de pipe te schrijven. Niettegenstaande dat zowel het ouderproces als het kindproces beide kunnen lezen en schrijven van/naar de pipe, wordt aangeraden om in beide processen de filedescriptor die niet gebruikt wordt te sluiten. Zoals reeds eerder aangegeven is de informatiestroom doorheen een pipe slechts in één richting, ttz. van het ouderproces naar het kindproces of omgekeerd. Is communicatie in beide richtingen nodig dan moet je twee pipes voorzien.

4. Schrijf een programma dat een getal neemt op de commandolijn en evenveel processen genereert als dat getal aangeeft. Binnen ieder kindproces wordt een willekeurig getal genereert en opgestuurd naar de het ouderproces. Het ouderproces bepaalt het grootste van de gegeneerde getallen bepaalt en vervolgens brengt het ieder kindproces op de hoogte van wie de winnaar is, ttz. welk proces het grootste getal heeft gegenereerd. De uitvoer voor zes kindprocessen ziet er als volgt uit:

```
Process 1819 is the winner
I'm the winner!
Process 1819 is the winner
Process 1819 is the winner
Process 1819 is the winner
Process 1819 is the winner
```